AI Unplugged

Migration handoff: Firebase Firestore + Storage → self-hosted Postgres + local filesystem (Firebase Auth retained)

PDF prepared by Claude (Anthropic Claude Code) for Anarv Vasavada · Date: 2026-05-08 · Status: code complete, smoke-tested locally, security review applied

1. Executive summary

The platform previously stored every record in **Firestore** and every uploaded file in **Firebase Storage**, while authenticating users with **Firebase Auth**. To remove vendor lock-in for data and storage and to give us full operational control, we migrated:

- **Database:** Firestore → **PostgreSQL** (managed via Prisma).
- **File storage:** Firebase Storage → **local filesystem** on the same server, served by the Node backend at `/uploads/<path>`.
- **Authentication:** **unchanged** — Firebase Auth on the client, ID-token verification on the backend via `firebase-admin`.

The frontend, which used to read from Firestore directly via the client SDK, now reads exclusively through the backend HTTP API (`/api/platform/*`). All file uploads now POST to the backend as multipart and are saved to disk.

2. Where the data lives — important

Development (right now): all application data is stored **on the developer's Mac** — the Postgres data files at `/opt/homebrew/var/postgresql@16/` and the uploaded files at `backend/uploads/`. If the dev machine is off, the site is offline; if the dev machine dies without a backup, the data is gone. **Local dev data is for development only.**

Production: the deployment server runs its own Postgres and its own `backend/uploads/` directory. The production database starts **empty** on first deploy. Both are backed up to `/var/backups/` by a daily cron and should be copied off-server periodically.

Firebase Authentication identities (emails, passwords, Google linkages) live in Firebase's cloud and are shared between dev and prod — that's the only piece we don't host.

What	Local development	Production server
Database files	`/opt/homebrew/var/postgresql@16/`	`/var/lib/postgresql/16/main/`
Uploaded files	`backend/uploads/` in the repo	`/var/www/ai-unplugged/backend/uploads/`
App code	Local clone of the repo	`/var/www/ai-unplugged/`
Service-account JSON	`backend/serviceAccount.json`	`/var/www/ai-unplugged/backend/serviceAccount.json`
Auth identities	Firebase cloud (shared)	

3. Architecture: before vs. after

	Before	After
Auth	Firebase Auth (web SDK + admin SDK)	**Same** — Firebase Auth retained
Database	Firestore — 11 collections, accessed both client- and server-side	PostgreSQL via Prisma — 13 tables, accessed only by the backend
File storage	Firebase Storage bucket	Local filesystem under `backend/uploads/`, served by Node at `/uploads/...`
Admin role	Firebase custom claim `admin: true`	Postgres column `users.role = 'ADMIN'`
Frontend data fetches	Direct Firestore queries from the browser	`fetch` against backend API endpoints
Deploy target	Firebase Hosting / Functions (planned)	Single Node process on a Linux server, fronted by nginx/Caddy

4. How to run it locally — exact commands

One-time prerequisites

```
# 1. Install Postgres 16 and start it
brew install postgresql@16
brew services start postgresql@16

# 2. Create the database (uses your macOS username as the role)
```

```
createdb aiunplugged

# 3. Backend env config
cd "AI Unplugged/backend"
cp .env.example .env       # then fill in DATABASE_URL, BREVO_*,
BOOTSTRAP_ADMIN_EMAILS
# Place the Firebase service-account JSON at backend/
serviceAccount.json

# 4. Backend deps + DB schema
npm install
npx prisma migrate dev --name init

# 5. Frontend deps
cd ../frontend
cp .env.example .env       # only the VITE_FIREBASE_* keys are
needed
npm install
```

Daily run (two terminals)

```
# Terminal 1 — Backend on :8000
cd "AI Unplugged/backend"
node server.js

# Terminal 2 — Frontend on :5173 (Vite proxies /api and /uploads
to :8000)
cd "AI Unplugged/frontend"
npm run dev
```

Open `http://localhost:5173`. Sign in. To become admin: ensure your email is in `BOOTSTRAP_ADMIN_EMAILS` in `backend/.env` AND verify the email in Firebase (Google sign-in is auto-verified — easiest path).

Stop the servers

```
pkill -f "node server.js"
pkill -f vite
brew services stop postgresql@16    # only if you want to stop
the DB too
```

5. Database

Property	Value
Engine	PostgreSQL 16
Database name	`aiunplugged`

Local connection string	`postgresql://<your-mac-user>@localhost:5432/aiunplugged`
Production connection string	`postgresql://aiu:<password>@localhost:5432/aiunplugged`
Schema source of truth	`backend/prisma/schema.prisma`
Migrations	`backend/prisma/migrations/` (committed to the repo)
Inspect tables	`psql aiunplugged -c '\dt'`
Common queries	`psql aiunplugged -c 'SELECT email, role FROM "User";'`

The 13 tables (12 application models + Prisma's internal migrations table):

Table	Purpose
User	Authenticated user. Primary key = Firebase UID. Stores role (USER / ADMIN), display name, newsletter preference.
NewsletterSubscriber	Email list with subscribe/unsubscribe state.
Event	Public events. Title, date, location, map fields, publish state, entry mode, form id.
EventForm	Form schemas (event + node-lead). JSON `fields` column.
EventRegistration	One per event sign-up. Stores answers JSON, review status, optional user FK.
NodeLeadApplication	Node-lead application form submissions.
HostApplication	Host-application submissions.
Skill	SkillDB markdown submissions: metadata + on-disk path + cached markdown.
Update	Blog/announcement posts. Body stored as JSON paragraphs and HTML.
Comment	Per-update comments with moderation status.
Resource	Resource cards/pages on the site.
SiteSetting	Key-value JSON store, e.g. home spotlight event IDs.
NewsletterCampaign	Audit log of sent newsletters.
_prisma_migrations	Prisma's internal migration history (do not modify).

6. File storage

- Files are written to `backend/uploads/` (configurable via `UPLOAD_DIR`).

- Path layout:
  - `skills/<userId>/<skillId>/<timestamp>-<name>.md`
  - `updates/<draftId>/attachments/<timestamp>-<name>`
  - `resources/<draftId>/media/<timestamp>-<name>`
- Served at `GET /uploads/<path>`. Path traversal rejected, `X-Content-Type-Options: nosniff` set, MIME type allow-listed on upload.
- Public URLs use `PUBLIC_BASE_URL` in production; empty in dev so the frontend hits the same origin via the Vite proxy.

7. Admin access & promotion

- Admin status lives in Postgres: `"User".role = 'ADMIN'`.
- The frontend treats a logged-in user as admin when the synced profile returns `role: "admin"` (lowercased on the wire). The admin dashboard is at `/admin`.
- **Bootstrap promotion:** on first sign-in, if the email matches an entry in `BOOTSTRAP_ADMIN_EMAILS` AND `email_verified === true` in Firebase, the user is auto-promoted to `ADMIN`. Use Google sign-in for instant verification.
- **Granting admin from the UI** (Admin Panel → "Add admin"): the target email must already belong to a registered user. They have to sign in once first; only then can you grant them admin. The backend cannot create an admin role for a UID that doesn't exist yet.
- **Manual promotion** (DB-direct, breaks glass scenarios only):
  `psql aiunplugged -c "UPDATE \"User\" SET role='ADMIN' WHERE email='someone@example.com';"`
- The user must reload (or sign out + back in) after promotion so the React state picks up the new role.

8. Backend API surface (added during migration)

Method	Path	Purpose
GET	/api/platform/events, /events/:id	Public events list and detail
POST / PUT / DELETE	/api/platform/events, /events/:id	Admin event CRUD
GET	/api/platform/updates, /updates/:slug	Public updates list and detail

POST	/api/platform/updates	Admin upsert of an update post
GET	/api/platform/resources, /resources/:slug	Public resources list and detail
POST / DELETE	/api/platform/resources, /resources/:id	Admin resource CRUD
GET / POST	/api/platform/event-forms, /event-forms/:id	Form schemas
GET / PUT	/api/platform/site-settings/:key	Settings key-value
GET	/api/platform/comments?updateId=…	Approved comments for an update
POST / GET / PUT / DELETE	/api/platform/comments, /admin/comments, /comments/:id	Comment submit + admin moderation
GET	/api/platform/event-registrations, /node-lead-applications, / subscribers	Admin lists
PUT	/api/platform/ {eventRegistrations,nodeLeadApplications,hostApplications}/:id/ review	Admin review-status update
GET / POST / PUT / DELETE	/api/platform/skills, /skills/:id, /skills/:id/review, /skills/:id/ downloads	SkillDB lifecycle
POST	/api/platform/uploads/update-attachment, /uploads/resource-image	Multipart upload (admin)
POST	/api/platform/auth/sync	Upsert user from Firebase token; auto-promotes **verified**

		bootstrap admins

All write endpoints require `Authorization: Bearer <Firebase ID token>`. Admin endpoints additionally check `users.role = 'ADMIN'` in Postgres.

9. Production deployment guide — for the colleague who will deploy

What you need before you start

1. A Linux server (Ubuntu/Debian) with sudo access. 2 GB RAM is comfortable.
2. A domain name pointing to the server (A record).
3. The Git repo URL.
4. The Firebase **service-account JSON** for the AI Unplugged Firebase project. Download from Firebase Console → Project settings → Service accounts → Generate new private key. Save as `serviceAccount.json`. Treat like a password.
5. A fresh Brevo API key (the previous one was rotated), the verified Brevo sender email, and the list of bootstrap admin emails.

Steps

```
# 1. System packages
sudo apt update
sudo apt install -y nodejs npm postgresql nginx certbot python3-
certbot-nginx

# 2. Postgres role + database
sudo -u postgres psql <<'SQL'
CREATE ROLE aiu LOGIN PASSWORD 'PICK_A_STRONG_ONE';
CREATE DATABASE aiunplugged OWNER aiu;
SQL

# 3. Code
sudo mkdir -p /var/www && sudo chown $USER /var/www
git clone <repo-url> /var/www/ai-unplugged
cd /var/www/ai-unplugged

# 4. Frontend build
cd frontend && npm ci && npm run build && cd ..

# 5. Backend deps
cd backend && npm ci

# Copy serviceAccount.json into backend/ (scp from your laptop)
```

```
# Create backend/.env (template below), then:
npx prisma migrate deploy
```

backend/.env for production

```
PORT=8000
DATABASE_URL=postgresql://aiu:PICK_A_STRONG_ONE@localhost:5432/
aiunplugged
UPLOAD_DIR=/var/www/ai-unplugged/backend/uploads
PUBLIC_BASE_URL=https://your-domain.com
FIREBASE_SERVICE_ACCOUNT_PATH=./serviceAccount.json
BREVO_API_KEY=<rotated key>
BREVO_SENDER_EMAIL=<verified sender>
BREVO_SENDER_NAME=AI Unplugged
BOOTSTRAP_ADMIN_EMAILS=anarv.work@gmail.com,colleague@example.com
```

systemd unit (`/etc/systemd/system/ai-unplugged.service`)

```
[Unit]
Description=AI Unplugged
After=network.target postgresql.service

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/ai-unplugged/backend
EnvironmentFile=/var/www/ai-unplugged/backend/.env
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo chown -R www-data:www-data /var/www/ai-unplugged
sudo systemctl daemon-reload
sudo systemctl enable --now ai-unplugged
sudo journalctl -fu ai-unplugged           # tail logs
```

nginx reverse proxy (`/etc/nginx/sites-available/ai-unplugged`)

```
server {
  listen 80;
  server_name your-domain.com;
  client_max_body_size 25M;

  location / {
    proxy_pass http://127.0.0.1:8000;
```

```
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

```
sudo ln -s /etc/nginx/sites-available/ai-unplugged /etc/nginx/
sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d your-domain.com    # Let's Encrypt HTTPS
```

Daily backup cron (`/etc/cron.daily/aiu-backup`)

```
#!/bin/sh
TS=$(date +%F)
sudo -u postgres pg_dump aiunplugged | gzip > /var/backups/aiu-
$TS.sql.gz
tar czf /var/backups/aiu-uploads-$TS.tar.gz -C /var/www/ai-
unplugged/backend uploads
find /var/backups -name 'aiu-*' -mtime +14 -delete
```

`sudo chmod +x /etc/cron.daily/aiu-backup`. Copy `/var/backups/` off-server periodically (S3, B2, rsync).

Redeploys (after code push)

```
cd /var/www/ai-unplugged
git pull
cd frontend && npm ci && npm run build
cd ../backend && npm ci && npx prisma migrate deploy
sudo systemctl restart ai-unplugged
```

Things the deployer must NOT do

- Do not commit `serviceAccount.json` or `.env` to Git (already gitignored).
- Do not change Firebase Auth — it's still the login system.
- Do not run `prisma migrate dev` on the server — only `migrate deploy`.
- Do not expose port 8000 publicly. Firewall it; nginx is the only public listener.
- Do not skip backups before the first real users sign up.

Health check after deploy

```
curl -s https://your-domain.com/api/platform/health
# expected: {"ok":true,"version":"local-...","diagnostics":{...}}
```

10. Security review & fixes applied

ID	Severity	Issue	Status
C1	**CRITICAL**	Logged-in user could create published `Update` posts via the comment endpoint.	**Fixed**
C2	**CRITICAL**	Bootstrap admin promotion didn't check `email_verified`.	**Fixed** — verified email now required.
C3	**CRITICAL**	`/api/platform/health` leaked filesystem paths and env-var inventory.	**Fixed** — public response stripped, full diagnostics gated to admins.
H1	**HIGH**	Uploads served without nosniff; SVG XSS path open.	**Fixed** — MIME allow-list on upload, nosniff + Content-Disposition on serve.
M2	**MEDIUM**	500-class errors leaked Prisma internals.	**Fixed** — generic message; trace logged server-side.
H2	**HIGH**	Admin list endpoints unpaginated.	Open — first patch.
H3	**HIGH**	Skill markdown rendering may allow stored XSS.	Open — sanitize on save or on render.
M1	**MEDIUM**	No app-level rate limiting.	Mitigated at nginx; app-level limiter recommended.
M3	**MEDIUM**	Hand-rolled multipart parser.	Open — replace with `busboy`.
M4	**MEDIUM**	Event registration silently re-subscribes user to newsletter.	Open — explicit consent checkbox.
M5	**MEDIUM**	Brevo API key was exposed during dev.	**Action required:** rotate in Brevo console, update `backend/.env`, restart backend.

11. Verification checklist

1. `psql aiunplugged -c '\dt'` shows the 13 tables listed in §5.
2. `curl https://your-domain.com/api/platform/health` returns `ok: true` with no filesystem paths in the body.
3. Sign in with Google → a `User` row appears with `role = 'USER'` initially.
4. With your email in `BOOTSTRAP_ADMIN_EMAILS` and email-verified, sign in → role flips to `ADMIN`; admin UI unlocks at `/admin`.
5. As admin: create + publish an event. Public events page lists it.

6. Submit an event registration → row in `"EventRegistration"`; confirmation email sent.
7. Upload a SkillDB `.md` file → file appears under `backend/uploads/skills/<uid>/…`; `GET /uploads/...` serves it with nosniff header.
8. Browser DevTools network tab: zero requests to `firestore.googleapis.com` or `firebasestorage.googleapis.com`.
9. Try POSTing a comment with a non-existent `updateSlug` — should now return `404`.

12. Configuration reference

Variable	Where	Purpose
DATABASE_URL	backend/.env	Postgres connection string.
UPLOAD_DIR	backend/.env	Where uploaded files live. Defaults to `./uploads`.
PUBLIC_BASE_URL	backend/.env	Origin used for absolute upload URLs in production.
FIREBASE_SERVICE_ACCOUNT_PATH	backend/.env	Path to Firebase service-account JSON.
BREVO_API_KEY / BREVO_SENDER_EMAIL / BREVO_SENDER_NAME	backend/.env	Transactional & campaign email.
BOOTSTRAP_ADMIN_EMAILS	backend/.env	Comma-separated list. Auto-promotes only when the user's email is verified in Firebase.
VITE_FIREBASE_*	frontend/.env	Firebase web SDK config (Auth only).

13. What to tell the team

- **Ownership:** we own our data now. Postgres on our server replaces Firestore; uploaded files live on the server's disk; Firebase is kept only for login.
- **No user-facing change:** URLs, sign-in flow, and admin workflows are unchanged.
- **Local dev:** Postgres must be running, then `node server.js` in `backend/` and `npm run dev` in `frontend/`. Two terminals.

- **Schema changes:** edit `backend/prisma/schema.prisma` → `npx prisma migrate dev --name <change>` → commit the migration. Server runs `npx prisma migrate deploy` on each release.
- **Admin role:** stored in Postgres `users.role`. To promote someone, they must sign in once first (Google sign-in is fastest).
- **Security:** three critical issues fixed pre-handoff. One operational action: rotate the Brevo API key.
- **Open follow-ups:** pagination on admin lists, app-level rate limiting, swap multipart parser for `busboy`, sanitize skill markdown, fix the silent newsletter re-subscribe.
- **Repo housekeeping:** `firebase.json`, `firestore.rules`, `firestore.indexes.json`, `backend/functions/` are dead code; delete in a follow-up.

14. Critical files for review

- `backend/server.js` — full HTTP API.
- `backend/prisma/schema.prisma` — schema source of truth.
- `backend/src/db.js`, `backend/src/storage.js` — Prisma singleton + FS helpers.
- `frontend/src/lib/platform.js`, `frontend/src/lib/skilldb.js` — frontend data access.
- `frontend/src/lib/firebase.js`, `frontend/src/context/AuthContext.jsx` — Auth-only Firebase client.
- `backend/.env.example`, `README.md`, `.gitignore`.

PDF prepared by Claude (Anthropic Claude Code) on behalf of Anarv Vasavada · 2026-05-08 · For internal review only.